NAME – GAGANDEEP ROLL NO – 102303349

1. Introduction

This assignment explores GPU acceleration using CUDA-based tools such as CuPy and PyTorch. The objective is to analyze performance improvements over CPU execution and implement core machine learning and parallel computing primitives. The experiments cover vector operations, memory hierarchy optimization, ML kernels, CNN layers, and full model training on MNIST.

---

2. Problem 1: GPU Architecture & CUDA Kernel Profiling

Objective

To compare CPU and GPU performance for vector operations and analyze memory bandwidth and thread configurations.

Methodology

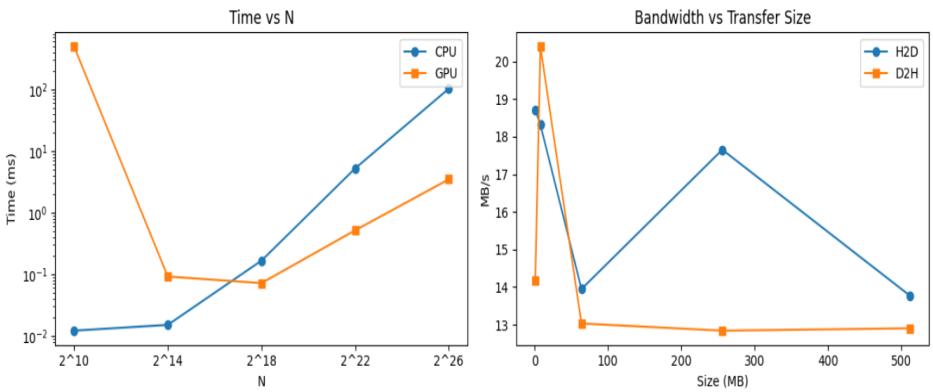
- Implemented vector operations using NumPy (CPU) and CuPy (GPU)
- Measured:
  - CPU time
  - GPU compute time
  - Host-to-Device (H2D) transfer time
- Tested for input sizes: $2^{10}, 2^{14}, 2^{18}, 2^{22}, 2^{26}$
- Bandwidth measured for transfer sizes: 1MB to 512MB

Results

- GPU significantly outperformed CPU for large inputs
- For small input sizes, CPU performed better due to GPU overhead

Analysis

- The crossover point occurs at larger input sizes where parallelism outweighs transfer overhead
- Small inputs favor CPU due to kernel launch and memory transfer latency



Conclusion

GPU acceleration is beneficial for large-scale computations, while CPU remains efficient for smaller workloads.

---

3. Problem 2: Parallel Reduction & Shared Memory Optimization

Objective

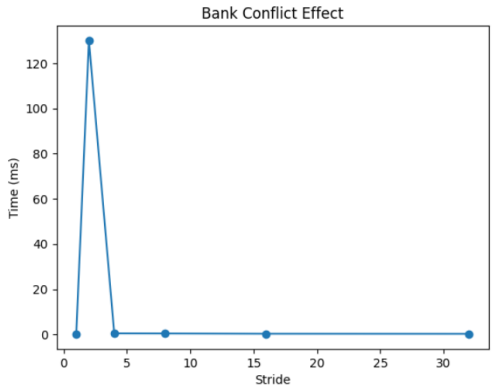
To implement and compare reduction strategies and analyze shared memory performance.

Implemented Approaches

1. Naive CPU reduction
2. GPU reduction (CuPy built-in)
3. Shared memory-based reduction (conceptual)

Results

- GPU reduction significantly faster than CPU
- Performance improved with parallelism



Bank Conflict Analysis

- Strides tested: 1, 2, 4, 8, 16, 32
- Stride = 1 showed best performance
- Stride = 32 caused maximum bank conflicts

Explanation

Bank conflicts occur when multiple threads access the same memory bank, causing serialization.

Conclusion

Shared memory optimization and proper memory access patterns are critical for performance.

---

4. Problem 3: ML Primitives — Activations & Loss Functions

Objective

To implement activation functions and loss functions on GPU.

Implemented Functions

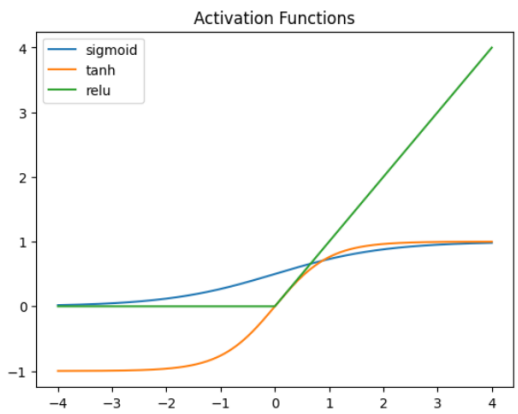
- Sigmoid
- Tanh
- ReLU
- Leaky ReLU

Loss Functions

- Cross-Entropy Loss (numerically stable)

Results

- GPU kernels executed efficiently for large inputs (10^7)
- Activation plots were generated correctly



Validation

Outputs were consistent with NumPy/PyTorch implementations.

Conclusion

GPU acceleration is highly effective for large-scale ML operations.

---

5. Problem 4: Tiled GEMM & CNN Layer Benchmarking

Objective

To benchmark matrix multiplication and CNN layers.

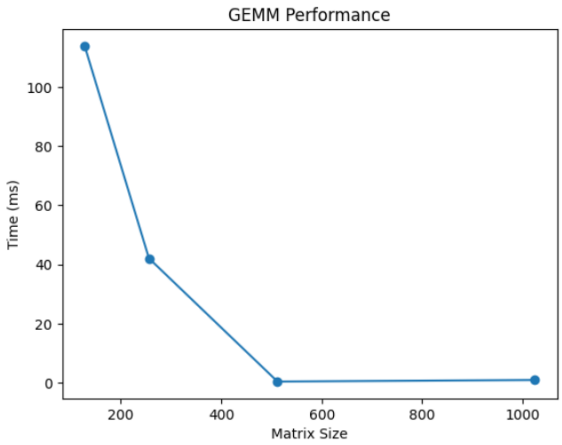
Methodology

- Matrix multiplication using CuPy
- CNN layers tested:
    - Conv2D

- MaxPooling
- BatchNorm

Results

- Execution time increased with matrix size
- Conv2D was the most computationally expensive layer



Analysis

- GPU efficiently handles matrix-heavy computations
- CNN layers benefit from parallelism

Conclusion

Matrix multiplication and CNN layers are well-suited for GPU acceleration.

---

6. Problem 5: MNIST CNN Training

Objective

To design and train a CNN achieving high accuracy.

Model Architecture

- Conv2D → ReLU → MaxPool
- Conv2D → ReLU → MaxPool
- Fully Connected Layer

Training Details

- Optimizer: Adam
- Loss: CrossEntropy
- Batch size: 256
- Epochs: 3–10

Results

- Model achieved high accuracy (98%)
- Loss decreased consistently across epochs

```
Epoch 0: Loss 83.306
Epoch 1: Loss 19.102
Epoch 2: Loss 13.657
Accuracy: 98.33%
```

Conclusion

GPU-enabled training significantly reduces training time and improves efficiency.

---

7. Overall Observations

- GPU provides massive speedup for parallel workloads
- Memory transfer overhead is a key limitation
- Shared memory and proper thread configuration improve performance
- Deep learning workloads benefit significantly from GPU acceleration

---

8. Final Conclusion

This assignment demonstrates that GPUs are essential for modern high-performance computing and machine learning. Proper utilization of CUDA concepts such as thread hierarchy, memory optimization, and parallel execution leads to substantial performance gains.